

Cahier des Charges Technique, Fonctionnel et Mathématique

Application E-commerce "Le Laboratoire" & Maximisation de la Fayda

Équipe de Direction

18 septembre 2026

Table des matières

1	Architecture Technologique & Infrastructure (Coût : 0 DA)	2
1.1	Stack Technique	2
1.2	Hébergement et Services Tiers	2
1.3	Sécurité	2
2	Ergonomie Mobile & Interface (PWA)	2
2.1	Contraintes PWA et Navigation	2
2.2	Optimisation de la Saisie (Fast-Input)	2
3	Modélisation Financière : "Le Laboratoire"	2
3.1	Variables d'Entrée du Test	3
3.2	Équations Fondamentales	3
4	Intelligence Artificielle & Maximisation de la Fayda	3
4.1	Fixation et Ajustement du Prix de Vente	3
4.2	Boucle de Feedback Dynamique (Pendant les 6 Jours)	3
5	Structure de la Base de Données (PostgreSQL / EF Core)	4

1 Architecture Technologique & Infrastructure (Coût : 0 DA)

L'application repose sur une architecture découplée pour garantir performance, sécurité et gratuité d'hébergement.

1.1 Stack Technique

- **Frontend** : React (Vite.js) configuré en PWA (Progressive Web App).
- **Backend / API** : C# ASP.NET Core Web API avec Entity Framework Core.
- **Base de Données** : PostgreSQL (Hébergement Neon.tech ou Supabase).
- **Module IA** : Python (CrewAI) couplé à des LLM locaux (Ollama : DeepSeek-R1 / Qwen).

1.2 Hébergement et Sécurité

- **Déploiement Frontend** : Vercel (synchronisation GitHub, HTTPS automatique, CDN mondial).
- **Déploiement Backend** : Render ou Koyeb (déploiement via Dockerfile).
- **Authentification** : JWT (JSON Web Token). Comptes limités à 2 Administrateurs, pré-générés en base de données (aucune inscription publique).

2 Gestion des Images (Upload et Stockage)

L'hébergement gratuit (Render) est volatil. Si les images étaient stockées directement sur le serveur backend, elles seraient définitivement perdues à chaque redémarrage (cold start) ou mise à jour de l'API.

2.1 Intégration de Cloudinary

- **Flux de données (Workflow)** :
 1. Lors de l'ajout d'un produit, l'administrateur prend la photo (via smartphone).
 2. Le frontend React ou l'API .NET envoie ce fichier image directement au service tiers **Cloudinary** via son API sécurisée.
 3. Cloudinary stocke l'image, l'optimise pour le web (compression automatique), et renvoie un lien public HTTPS (ex : `res.cloudinary.com/.../sel3a.jpg`).
 4. L'API .NET enregistre **uniquement ce lien URL** sous forme de texte dans la base de données PostgreSQL.
- **Avantage** : Zéro charge sur la base de données, chargement ultra-rapide des images côté client, et préservation totale des données même si le serveur API redémarre.

3 Ergonomie Mobile & Interface (PWA)

Le second administrateur opérant exclusivement sur smartphone, l'interface Frontend adopte une approche "Mobile-First" stricte.

3.1 Contraintes PWA et Saisie Rapide (Fast-Input)

- **Installation native** : Fichier `manifest.json` et Service Worker pour installation sur l'écran d'accueil iOS/Android, avec barre de navigation inférieure (Bottom Navigation Bar).

- **Clavier Numérique Natif** : Utilisation des attributs HTML (`inputMode="decimal"`) pour forcer le pavé numérique lors de la saisie des dépenses Ads et des revenus.
- **Capture Photo Directe** : Intégration de l'attribut `capture="environment"` sur le champ d'upload de produits pour déclencher l'appareil photo arrière directement depuis l'application web.

4 Modélisation Financière : "Le Laboratoire"

Le Laboratoire teste la rentabilité d'une nouvelle marchandise (*sel3a*) avant achat en gros.

4.1 Variables d'Entrée du Test

- **Capital Stock (*Ras Lmal*)** : Quantité initiale (ex : 8 pièces) et coût total d'achat fournisseur.
- **Budget Sponsoring (Ads)** : Budget total alloué sur la page officielle (Facebook/Instagram). Défaut : 2 000 DA sur 6 jours.
- **Frais de Logistique** : Le "ticket de bureau" payé pour chaque commande expédiée. Défaut : 15 DA.

4.2 Équations Fondamentales

Le Backend .NET calcule en temps réel :

$$\text{Dépense Totale} = \text{Ras Lmal} + \text{Total Ads Consommé} + (\text{Ventes Confirmées} \times \text{Ticket Bureau}) \quad (1)$$

$$\text{Revenu Total} = \text{Ventes Confirmées} \times \text{Prix de Vente Unitaire} \quad (2)$$

$$\text{Fayda (Bénéfice Net)} = \text{Revenu Total} - \text{Dépense Totale} \quad (3)$$

5 Intelligence Artificielle & Maximisation de la Fayda

Le système intègre un pipeline IA multi-agents (CrewAI + Ollama) pour optimiser les prix et analyser le marché.

5.1 Méthode technique de Scraping (Facebook, Instagram, TikTok)

Les réseaux sociaux bloquent agressivement les robots de scraping direct. L'Agent IA contourne ce problème via une méthode indirecte :

- **Recherche par Google Dorking** : L'Agent Python utilise une API de recherche (ex : SerpApi) ou Playwright pour interroger le moteur de recherche Google avec des opérateurs ciblés. Exemple de requête automatisée :
`site:facebook.com OR site:instagram.com OR site:tiktok.com "Nom de la sel3a" "DA" Algérie.`
- **Extraction Sémantique par LLM** : Les résultats bruts textuels renvoyés par la recherche sont envoyés au modèle LLM local (Ollama). Le modèle a pour instruction (Prompt) de lire ce texte bruité, d'en extraire le prix exact en Dinars Algériens, et de structurer la donnée en JSON.
- **Définition du Prix Plafond** : Le système compile ces extractions, élimine les prix aberrants, et définit le prix moyen du marché local. Ce prix devient la limite psychologique à ne pas dépasser.

5.2 Ajustement Dynamique et Arrondi Psychologique

- **Arrondi Psychologique** : Le modèle .NET ajuste automatiquement le prix de vente brut au seuil psychologique le plus attractif (ex : proposition à 2 900 DA plutôt que 3 000 DA).
- **Boucle de Feedback (6 Jours)** :
 - *Alerte Prix Trop Haut* : CTR élevé ($> 2\%$) mais 0 vente au Jour 3 $\rightarrow$ Recommandation d'une baisse immédiate du prix pour débloquer les conversions.
 - *Opportunité de Maximisation* : Écoulement rapide du stock avec un faible CPA $\rightarrow$ Recommandation d'augmentation du prix sur le stock restant.
 - *Arrêt d'Urgence* : Plus de 60% du budget Ads consommé sans vente $\rightarrow$ Alerte rouge, recommandation de stopper la campagne pour protéger le reste du budget.

6 Structure de la Base de Données (PostgreSQL / EF Core)

- **Format JSONB** : Utilisation du type jsonb pour la colonne Specifications de la table Products, permettant d'ajouter dynamiquement des attributs (couleurs, tailles) sans modifier le schéma de la base.
- **Entités Principales** : Users, Products, ProductTests, TestDailyMetrics.